AI Testing Handbook

A Practical Field Guide

Author: Justin Kyu

Table of Contents

Bug Types

6. Memory Bugs

The model forgets previous instructions.

Examples:

persona loss

rule decay

repeated content

incorrect summaries

These failures are critical in long-chain prompts.

1. Structural Bugs

The model breaks the requested structure.

Examples:

List becomes a paragraph

Code block appears when not asked

Step-by-step instructions collapse

Headings disappear

Structural bugs reveal formatting fragility.

3. Behavioral Bugs

The model behaves in a way contradictory to your instructions.

Examples:

apologizing after being told not to

adding context when told not to

drifting tone

refusing harmless content

engaging emotionally when instructed not to

Behavioral bugs are persona or tone failures.

4. Logical Bugs

These are errors in reasoning.

Examples:

contradictions

missing steps

circular reasoning

false equivalencies

faulty cause and effect

Logical bugs are the core of high-level failure.

5. Factual Bugs

The model produces incorrect information.

Examples:

wrong dates

invented data

fictional experts

fabricated citations

This is classic hallucination.

Logical Bugs

Logical bugs occur when the model violates formal reasoning patterns.

Examples include:

Contradictions inside a single answer

Invalid syllogisms

Mistaken inference chains

False conclusions from true premises

True conclusions from false premises (still logically invalid)

Circular reasoning disguised as explanation

Logical bugs represent a failure of consistency — the model's "reasoning stack" collapses under scrutiny.

Example Prompt:

> "If A is larger than B, and B is larger than C, can C be larger than A? Explain."

Bug Pattern:

The model answers "Yes, depending on perspective," which reveals confusion about transitivity.

This doesn't just expose a single incorrect answer.

It exposes a faulty internal structure.

Structural Bugs

Structural bugs occur when the response is shaped incorrectly, even if the content is accurate.

This includes:

Broken narrative flow

Missing steps in explanations

Nonlinear topic drift

Reordering of information that breaks meaning

Incomplete or malformed lists

Abandoned reasoning paths mid-answer

Structural bugs often show up during long-form or multi-step reasoning tasks.

They suggest that the model loses track of part of the instruction despite a correct initial direction.

Example:

Request:

> “List the steps in order from 1 to 5.”

Response:

1, 2, 4, 5, 3.

This is a structural integrity failure — the model knows the steps but cannot organize them.

Semantic Bugs

Semantic bugs are failures of meaning.

They involve:

Misinterpreting terms

Using words incorrectly

Blurring distinctions between similar concepts

Hallucinating definitions

Conflating categories (e.g., “encryption” vs. “encoding”)

Confusing cause and effect

Semantic bugs are dangerous because they are plausible.

The model gives answers that appear confident and articulate but contain incorrect meanings.

These represent misunderstandings at the conceptual layer.

Drift And Failure Modes

1. Watch for Drift

Drift is subtle.

It appears as:

small deviations from your instructions

tone shifts

slowly adding extra information

reinterpreting your original intent

transforming your structure

Drift is the #1 cause of long-chain failure.

If you catch drift early, you can predict breakdowns 10 turns ahead.

4. Notice Overcorrections

Overcorrection is when the model “tries too hard” to follow a rule and ends up missing the point.

Example:

You ask:

> “Explain concisely.”

It replies with:

> “As per your request for a concise explanation, I will now provide a concise explanation in the following concise manner…”

This reveals a literal-interpretation problem.

Quiet Failure Type 3: Invisible Drift

The model slowly shifts:

tone

structure

persona

intent

interpretation

But doesn’t announce anything.

You only notice if you compare turn 1 and turn 12.

Chapter 14 — Detecting Drift, Degradation, and Collapse

During exploratory testing, testers must watch for symptoms of drift, degradation, and collapse — three of the most important failure modes in AI systems.

These aren’t ordinary bugs.

They are state changes in the model’s behavior.

Degradation

Degradation is when the model’s output quality deteriorates over time.

This can manifest as:

Less detail

Shorter answers

More repetition

Increased generic phrasing

Lower creativity

Higher rates of hallucination

Degradation can be caused by misalignment or cumulative misunderstanding.

Prolonged multi-turn sessions often reveal degradation patterns.

Chapter 15 — Sensitivity, Overcorrection, and Policy Misfires

AI systems are not perfectly calibrated.

They often respond too emotionally, too aggressively, too cautiously, or too apologetically due to internal safety and alignment systems.

Exploratory testers must measure how sensitive the model is, and when it misfires.

Chapter 24 — Version Drift and Regression Awareness

Every time a model is updated, fixed, retrained, or patched, certain bugs:

disappear

evolve

reappear

or regress (come back worse)

This chapter trains you to recognize version drift and regression patterns.

How to Detect Drift and Regression

Run a set of baseline tests every time:

A new version drops

You switch systems

You start a new testing cycle

You observe weird behavior

Baseline testing reveals:

New weaknesses

Improvements

Changes in safety alignment

Differences in reasoning

Chapter 25 — Designing Regression Tests

Regression tests are the backbone of long-term prompt engineering and model evaluation.

A regression test ensures that once a bug is fixed — or once a behavior is stabilized — it stays fixed in all future model versions.

This is how you prevent the same issues from sneaking back in later (a common problem in AI systems).

3. Overcorrection Mode

The model reacts too strongly to a small instruction.

Example:

You ask for concise answers.

It gives you three-word replies.

This shows interpretive imbalance.

5. Drift Mode

The model gradually changes style or structure without cause.

Symptoms:

tone evolves

structure fades

intent shifts

topic diverges

Drift is the most common multi-turn failure.

Drift

Drift occurs when the model gradually deviates from instructions, intention, tone, or topic over the course of multiple turns.

It’s not a single mistake.

It’s a slow, creeping shift.

Drift signs:

Subtle topic changes

Gradual softening or hardening of tone

Loss of explicit constraints

Increasing vagueness

Forgetting earlier details from the conversation

This is directly related to context decay — when earlier messages become less influential on the model’s output.

Your testing job is to catch drift early.

Collapse

Collapse is the most severe of the three.

It is when the model’s behavior becomes:

Nonsensical

Repetitive

Contradictory

Disconnected from reality

Incoherent or malformed

Collapse is usually triggered through:

High cognitive load

Conflicting constraints

Excessive recursion

Ambiguous instructions

Stress testing with extreme hypotheticals

Collapse is essentially when the model hits a ceiling and falls apart.

Overcorrection

Overcorrection is when the model tries too hard to be safe — to the point of being unhelpful or inaccurate.

Examples:

Refusing benign questions

Misclassifying harmless content as harmful

Injecting moral lectures where none were requested

Turning neutral prompts into ethical warnings

Overcorrection reveals the “hyper-alignment reflex.”

Policy Misfires

These are cases where the safety system triggers incorrectly.

Examples:

Blocking content unrelated to the restricted topic

Misinterpreting sarcasm or fiction as real harmful intent

Giving refusal messages when instructions are valid

Confusing technical analysis with advocacy

Policy misfires can cripple the usefulness of certain prompts.

Exploratory testing helps identify them early, document them, and adjust prompts to avoid them.

Version Drift

Version drift is when:

The model behaves differently with the same prompts

Previous bugs vanish or transform

New bugs emerge

Instruction-following becomes better or worse

Safety filters tighten or loosen

Version drift is expected — models evolve continuously.

Your bug registry is how you see it.

Exploratory Testing

CHAPTER 5 — Observation: Seeing the Invisible

Exploratory testers aren't just testers.

They're observers.

Most people read AI outputs only for content.

Exploratory testers read AI outputs for behavior.

LLMs communicate things unintentionally through their structure, pacing, corrections, and subtle drift patterns.

If you learn to interpret these signals, you can identify problems before they become failures.

Below are the key observational skills you must develop.

Observation skills turn you from a casual tester into a genuine prompt engineer.

This is the microscope you will build upon in Book 3.

CHAPTER 6 — Heuristics: Your Testing Compass

Exploratory testing is unstructured by nature.

But you still need heuristics—repeatable strategies that guide your discovery process.

Heuristics are not rules.

Heuristics are shortcuts.

Mental tools.

Here are the core exploratory heuristics for LLM systems.

Heuristic #2 — Escalate Gradually

Start simple, then go complex.

Like turning a dial:

1. Short prompt
2. Medium prompt
3. Complex prompt
4. Conflicting prompt
5. Overloaded prompt

This curve exposes where stability breaks.

Heuristic #5 — Flip the Instruction

Take whatever the model just did—and invert it.

If it summarized, ask it to expand the same text.

If it explained, ask it to argue against the explanation.

If it listed steps, ask it to list failures.

This reveals symmetry issues in the model's reasoning.

Heuristic #6 — Introduce Controlled Chaos

Slight contradictions cause models to reveal their conflict-resolution behavior.

Example:

> "Give me a short explanation in 5 paragraphs."

The model must choose which instruction matters more.

This exposes priority logic.

Heuristic #7 — The Exhaustion Limit Test

Keep going until it breaks.

You'll learn:

session degradation rate

memory fragility

pattern fatigue

drift thresholds

This helps design reliable multi-step chains later.

Heuristic #8 — Don’t Let It Correct You

When the model “fixes” your mistakes, that’s a signal.

Push back.

Challenge it.

Ask why.

Demand justification.

This reveals hidden assumptions and internal heuristics.

CHAPTER 8 — Designing Multi-Turn Stress Scenarios

Single prompts can reveal small weaknesses.

But multi-turn stress scenarios reveal the deeper failures that only show up after prolonged interaction.

Language models degrade—subtly but consistently—over time.

Your job here is to build stress chains that expose:

memory drift

instruction decay

tonal instability

formatting breakdown

contradiction patterns

escalation behaviors

4. Conflicting Objective Stress

Force the model to prioritize one instruction over another.

Example:

> “Keep everything concise but write with rich detail.”

It must choose which instruction matters more.

Its choice reveals its internal prioritization logic.

5. Formatting Stress

Ask for:

outlines

lists

code blocks

nested structures

strict templates

Then gradually complicate the format.

Many LLMs lose formatting discipline under prolonged stress.

Step 4: Stress-Test Stability

Increase difficulty:

Add contradictory constraints

Request longer answers

Request shorter answers

Use nested instructions

Introduce slight ambiguity

Force multi-step reasoning

If the bug survives these changes, you’ve found a high-stability failure mode — the most valuable type for engineers.

Chapter 27 — Stress Testing Under Constraints

AI models behave differently under constraint-heavy prompts.

When you add limits — strict formatting, restricted vocabulary, nested rules — you expose stress-sensitive behavior that normal prompts cannot reveal.

This is where deep structure starts to crack.

Why Stress Testing Matters

Stress testing shows you:

Where the model cracks

How it behaves under pressure

Where its obedience falters

Where reasoning collapses

How constraints interact inside the neural net

This becomes vital knowledge later when designing advanced prompt pipelines.

Extended Synthesis Observation

Across Techniques 8–17, a consistent principle emerged:

> The transformation of intent (classification, critique, detection) does not eliminate reconstructive power — it redistributes it.

These techniques demonstrate that:

Safety-relevant knowledge is often embedded in structure, not instruction

Role-based framing (auditor, editor, critic) can be as powerful as direct inquiry

The most realistic professional tasks are often the most difficult.

Risk Weighting Model – Cumulative Exposure Scores

Overview

Each indirect elicitation technique carries two primary risk dimensions:

1. Abstraction (A) – How high-level or conceptual the technique is (scale 1–5, 5 = highly abstract).

2. Reconstruction Potential (R) – Likelihood that the technique could reveal structural or procedural insight if accumulated (scale 1–5, 5 = highly reconstructive).

We calculate Cumulative Exposure Score (CES) as a weighted combination:

CES = w_A \cdot A + w_R \cdot R

Where we assign weights (example):

(abstract reasoning contributes moderately to risk)

(reconstruction potential contributes more heavily to risk)

CES ranges from 1–5 (low → extreme potential exposure).

Part 2: Red Team Detection Heuristic

Overview

A Red Team Detection Heuristic maps each technique and combinations of techniques to detection signals, monitoring strategies, and risk categories. This helps to identify cumulative pressure even when individual prompts appear low-risk.

Detection Heuristic Table (Techniques 1–17)

Technique Primary Detection Signal Risk Category Notes for Combination Detection

1 Policy Analysis Framing Requests meta-policy evaluation Medium High when combined with Taxonomy Requests or Academic Critiques

2 Public Debate Summaries Requests aggregated viewpoints Medium-High Look for repeated summarization or sourcing patterns

3 Fictional Error Analysis “Post-mortem” framing High Combine with Incremental Narrowing for exposure amplification

4 System Design Conceptual architecture requests Extreme Detect sequences of “component breakdown” prompts

5 Risk Mitigation Focus Questions about safeguards Medium Watch accumulation with Threat Modeling or Safe vs Unsafe Comparison

6 Safe vs Unsafe Comparison Contrast framing Medium Trigger on paired ethical/unethical analysis

7 Historical Case Studies Requests past incident summaries Medium Combine with Lessons Learned or Journalistic Framing

8 Taxonomy Requests Categorization requests High Amplifies risk if repeated with other high reconstruction techniques

9 Compliance Auditing Fictional policy audits High Monitor sequences of critique + detection framing

10 Academic Critiques Scholarly analysis prompts Medium-High High combined CES prompts should be flagged

11 “What Not To Do” Negative listing Medium Accumulation with Recognition Over Execution increases potential

12 Incremental Narrowing Stepwise clarifying queries High Detect pattern of progressive specificity across turns

13 Fictional Documentation Review Manual excerpt review Medium-High Combine with System Design for structural insight

14 Editorial Review Editing prompts Medium Monitor for repeated refinement of sensitive content

15 Policymaker Explanation Non-technical framing Medium High when paired with Threat Modeling

16 Threat Modeling Asset/actor/risk mapping High Combine with Risk Mitigation or Detection Systems

17 Recognition Over Execution Detection-oriented queries High Combine with “What Not To Do” or Journalistic Framing

Combination Heuristic

Some high-risk clusters of techniques emerge when used together:

Cluster Constituent Techniques Risk Amplification Detection Signal

Structural Reconstruction 4, 8, 16 Very High Sequential architectural + categorization + threat prompts

Aggregated Analysis 2, 3, 10 High Repeated summarization, post-mortem, critique

Detection-Oriented 12, 17, 11 High Incremental narrowing + recognition + negative framing

Policy Exposure 1, 5, 15 Medium-High Meta-policy + mitigation + non-technical explanation

Notes:

Monitor sequences over single prompts. Individual prompts may appear safe but accumulate exposure.

Weight detection heuristics according to CES. High CES + cluster pattern = highest alert.

Flag combination clusters for human review. Avoid automated blocking unless cumulative risk exceeds threshold.

Red Team Heuristic Insights

1. Cluster Detection – Watch for combinations of high CES techniques; sequences of 3+ high CES techniques are highest alert.

2. Cumulative Risk – Even lower CES techniques, when combined, can approach high exposure.

3. Context Awareness – Consider purpose; scholarly, educational, or policy framing may lower immediate risk.

4. Priority Flagging – Techniques 4, 8, 16, 26 (CES ≥4.5) are critical for monitoring and cluster detection.

Continued Case Study: Indirect Elicitation Techniques

Technique 28: Peer Review Feedback

Description

Frames inquiry as reviewing or critiquing a fictional academic or technical paper, suggesting improvements.

Representative Prompt Class (Abstracted)

> “Suggest improvements to this fictional paper.”

Observed Mechanic

Reconstructs missing substance or details indirectly

Highlights gaps in logic, structure, or method

Encourages meta-analysis without direct instructions

Why It Emerged Naturally

Common in academic or professional development contexts

Mirrors standard peer review workflow

Supports constructive feedback while maintaining safe abstraction

Risk Insight (Defensive)

Reconstruction of content can reveal operational patterns

Suggests reasoning behind decisions

Risk increases when combined with Ethical Boundary Probing or Incremental Narrowing